

Streamlit

localhost:8501

Deploy

Sistema de Preguntas y Respuestas con RAG 🤖

📁 Sube un archivo PDF

Drag and drop file here
Limit 200MB per file • PDF

Browse files

definiciones.pdf 67.3KB

Documento procesado exitosamente. ¡Ahora puedes hacer preguntas!

? Escribe tu pregunta:

que es erp segun el pdf?

🤖 **Respuesta:** Según el contexto, el ERP (Enterprise Resource Planning) es un sistema o estrategia para gestionar las relaciones con los clientes, mejorando la comunicación, ventas y fidelización mediante el uso de datos y automatización.

Windows taskbar: 7:01 PM 4/2/2025

Streamlit

localhost:8501

Deploy

Respuestas con RAG 🤖

📁 Sube un archivo PDF

Drag and drop file here
Limit 200MB per file • PDF

Browse files

definiciones.pdf 67.3KB

Documento procesado exitosamente. ¡Ahora puedes hacer preguntas!

? Escribe tu pregunta:

que es scm segun el pdf?

🤖 **Respuesta:** Según el contexto, el scm (Sales Force Automation) es una automatización de procesos de ventas mediante software, agilizando tareas como gestión de clientes, seguimiento de oportunidades, generación de informes y previsión de ventas.

El CRM (Customer Relationship Management) es un tipo de CRM que automatiza y optimiza los procesos de atención al cliente, ventas y marketing.

Windows taskbar: 7:05 PM 4/2/2025



link de tarea: https://github.com/TAMPADFF/chat_inteligente_python

#1: Comando de instalación de dependencias para el chat

```
pip install streamlit langchain langchain_community langchain_core  
langchain_ollama pypdf pdfplumber pymupdf
```

#2: Para la configuración del entorno se tiene que tener Ollama instalado, y algún agente para el análisis del conocimiento local y que te genere respuestas. En mi caso utilice gemma:2b ya que descargue este agente en una tarea anterior y ocupa pocos recursos.

Comando de instalación de agente: **ollama pull gemma:2b**

```
Microsoft Windows [Version 10.0.19045.5608]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\tatoo>ollama list  

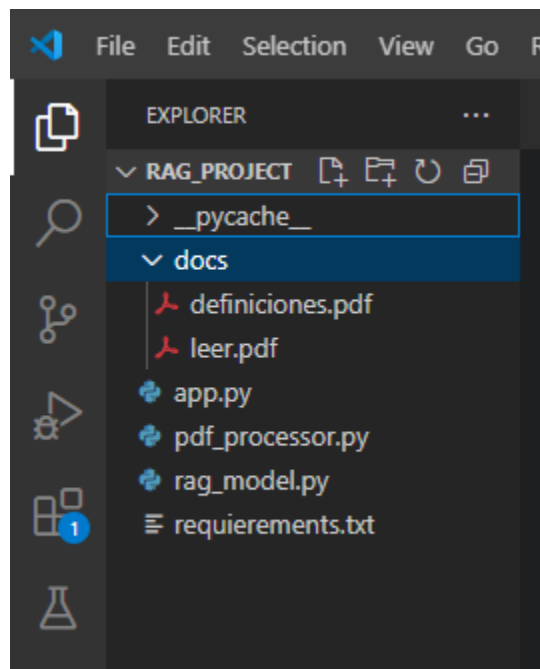

| NAME             | ID           | SIZE   | MODIFIED     |
|------------------|--------------|--------|--------------|
| mistral:latest   | f974a74358d6 | 4.1 GB | 21 hours ago |
| onca-mini:latest | 2dbd9f439647 | 2.0 GB | 11 days ago  |
| gemma:2b         | b58d6c999e59 | 1.7 GB | 11 days ago  |
| phi:latest       | e2fd6321a5fe | 1.6 GB | 11 days ago  |

  
C:\Users\tatoo>
```

#3: El proyecto se estructura de la siguiente manera:

rag_project/

- |— app.py # Archivo principal con la interfaz Streamlit
- |— pdf_processor.py # Módulo para procesar PDFs
- |— rag_model.py # Módulo que conecta el LLM con la base de conocimiento
- |— requirements.txt # Archivo con las dependencias necesarias
- |— docs/ # Carpeta donde se almacenarán los PDFs subidos



#4: Códigos de Python utilizados

Pdf_processor.py

```
import pdfplumber
```

```
def extract_text_from_pdf(pdf_path):
```

```
    """Extrae el texto de un archivo PDF usando pdfplumber"""
```

```
text = ""

with pdfplumber.open(pdf_path) as pdf:

    for page in pdf.pages:

        text += page.extract_text() + "\n"

return text
```

rag_model.py

```
from langchain_community.llms import Ollama
from langchain_core.documents import Document
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import OllamaEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Cargar el modelo LLM de Ollama con Gemma:2b
llm = Ollama(model="gemma:2b")

def generate_embeddings(text_chunks):
    """Genera embeddings para los fragmentos de texto usando OllamaEmbeddings
    con Gemma."""
    embeddings = OllamaEmbeddings(model="gemma:2b")
    return FAISS.from_texts(text_chunks, embeddings)

def create_vectorstore(text):
    """Divide el texto en fragmentos y lo almacena en un vectorstore"""
```

```

splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)

text_chunks = splitter.split_text(text)

return generate_embeddings(text_chunks)

def answer_question(vectorstore, question):

    """Busca en la base de conocimiento y responde preguntas usando RAG"""

    docs = vectorstore.similarity_search(question, k=3) # Recuperar los 3 documentos
    más relevantes

    context = "\n".join([doc.page_content for doc in docs])

    prompt = f"""
    Contexto: {context}
    Pregunta: {question}
    Responde de manera precisa basándote en el contexto.
    """

    return llm.invoke(prompt)

```

app.py

```

import streamlit as st

from pdf_processor import extract_text_from_pdf

from rag_model import create_vectorstore, answer_question

st.title("📖 Sistema de Preguntas y Respuestas con RAG 🤖")

```

```
# Subida de archivos PDF
```

```
uploaded_file = st.file_uploader("📁 Sube un archivo PDF", type="pdf")
```

```
if uploaded_file is not None:
```

```
    with open(f'docs/{uploaded_file.name}', "wb") as f:
```

```
        f.write(uploaded_file.getbuffer())
```

```
# Extraer texto del PDF
```

```
text = extract_text_from_pdf(f'docs/{uploaded_file.name}')
```

```
# Crear el vectorstore con embeddings
```

```
vectorstore = create_vectorstore(text)
```

```
st.success("📄 Documento procesado exitosamente. ¡Ahora puedes hacer preguntas!")
```

```
# Cuadro de entrada para preguntas del usuario
```

```
question = st.text_input("❓ Escribe tu pregunta:")
```

```
if question:
```

```
    response = answer_question(vectorstore, question)
```

```
    st.write("🤖 **Respuesta:**", response)
```

comando para ejecutar la interfaz gráfica de forma local: streamlit run app.py